Modul 2

# Vertiefung Datenbankentwurf

## Abbildung Generalisierung/Spezialisierung

Josef Altmann




Der vorliegende Foliensatz basiert auf:

Kemper, A., Leis V., Eickler, A.: *Datenbanksysteme,* Oldenbourg, 11. Auflage (2026)
Saake, G., Sattler, K.-U., Heuer, A.: *Datenbanken: Konzepte und Sprachen*, MITP, 6. Auflage (2018)
M. Blaha: UML Database Modeling Workbook, Technics Publications, LLC, 2013.
D. C. Hay: UML&Data Modeling, Technics Publications, LLC, 2011.

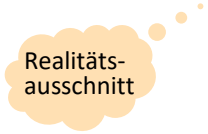
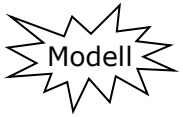
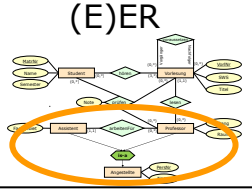
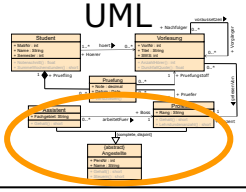
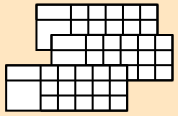
# Überblick

- Datenbankentwurf
  - Abbildung Entitätstypen
  - Abbildung Beziehungstypen
  - Abbildung Generalisierung
- Relationale Modellierung der Generalisierung
  - Abbildungsstrategien/-varianten
    - Basisrelationenmodell
    - Partitionierungsmodell
    - Volle Redundanz
    - Einrelationenmodell
  - Vergleich der Abbildungsstrategien/-varianten

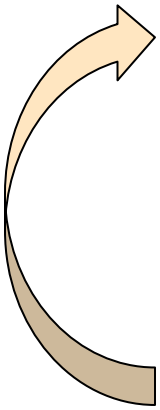
© 2026

# Datenbankentwurf

## Umsetzung konzeptuelles in logisches (relationales) Schema

| Phase | Modell | Ergebnis |
|---|---|---|
| Anforderungs-analyse | Requirements Engineering | Pflichtenheft (Use-Cases, ...)   **ANF 5.4:** Eine Vorlesung wird durch eine eindeutige Vorlesungsnummer identifiziert. ... |
| Konzeptueller Entwurf | Entity Relationship (UML, ...) | Konzeptuelles DB-Schema   (E)ER  UML  |
| Logischer Entwurf | Datenbankmodell (hierarchisch, objekt-orientiert, objekt-relational, semi-strukturiert, ...) | Logisches DB-Schema  `Vorlesung{VorlNr, Titel, ...}` `Studenten{MatrNr, Name, ...}` |
| Physischer Entwurf | Data Definition Language | Physisches DB-Schema   |

```
CREATE TABLE Vorlesung(
    VorlNr NUMBER(10) PRIMARY KEY,
    Titel VARCHAR(20),
    ...
);
```



**Realisierungsmodell**
**Relationale Modellierung der Generalisierung**

# Abbildung konzeptuelles in logisches Schema

## Abbildung Entitätstypen (Klassen)

- **Unabhängige Entitätstypen:**
  - Entitätstyp: Für jeden unabhängigen Entitätstyp *E* wird eine Relation *R* erstellt
  - Attribute: Attribute von *R* sind alle einfachen Attribute von *E*
  - Primärschlüssel: Ein Schlüsselattribut (kann auch zusammengesetzt sein) von *E* wird als Primärschlüssel für *R* ausgewählt
- **Existenzabhängige Entitätstypen:**
  - Existenzabhängiger Entitätstyp: Für jeden existenzabhängigen Entitätstypen *W* mit übergeordnetem Entitätstypen *E* wird eine Relation *R* erstellt.
  - Fremdschlüssel: Der Primärschlüssel der Relation des übergeordneten Entitätstypen *E* wird als Fremdschlüssel zu *R* hinzugefügt.
  - Primärschlüssel von *R* besteht aus der Kombination der
    - Primärschlüssel der übergeordneten Entitätstypen
    - des partiellen Schlüssels des existenzabhängigen Entitätstypen

© 2026

# Abbildung konzeptuelles in logisches Schema

## Abbildung Beziehungstypen (Assoziationen)

**Drei Ansätze** für Beziehung zwischen Entitätstypen *S* und *T*:

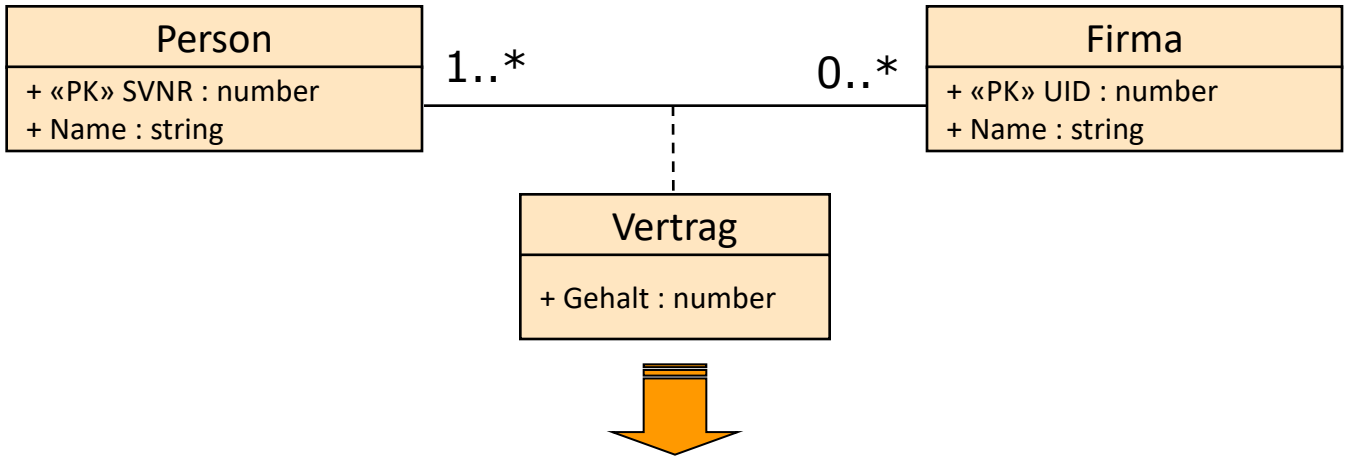
- **Zusammengefasste Relationen:** nur für 1:1
  - beteiligte Entitätstypen werden in einzige Relation zusammengelegt
  - keine Nullwerte falls *S* und *T* totale Beziehung eingehen
- **Fremdschlüssel:** 1:1, 1:N
  - eine der beteiligten Entitäten wird ausgewählt, z.B. *S* (N-Seite im Falle von 1:N)
  - Primärschlüssel von *T* wird als Fremdschlüssel zu *S* hinzugefügt
  - keine Nullwerte falls *S* totale Beziehung eingeht
- **Neue Beziehungsrelation:** 1:1, 1:N, M:N
  - neue Relation *R* mit den Primärschlüsseln von *S* und *T* als Fremdschlüssel
  - Primärschlüssel:
    - 1 : 1-Beziehung: einer der beiden Fremdschlüssel
    - 1 : N-Beziehung: Fremdschlüssel der N-Seite
    - M : N-Beziehung: beide Fremdschlüssel
  - keine Nullwerte

© 2026

# Abbildung konzeptuelles in logisches Schema

## Abbildung Beziehungstypen – Assoziationsklasse

Beziehung wird im Falle einer Beziehungsklasse durch eine eigene Relation dargestellt mit den Attributen der Beziehungsklasse




**Logischer Entwurf:**

Person: {[SVNR:number, Name:string]}

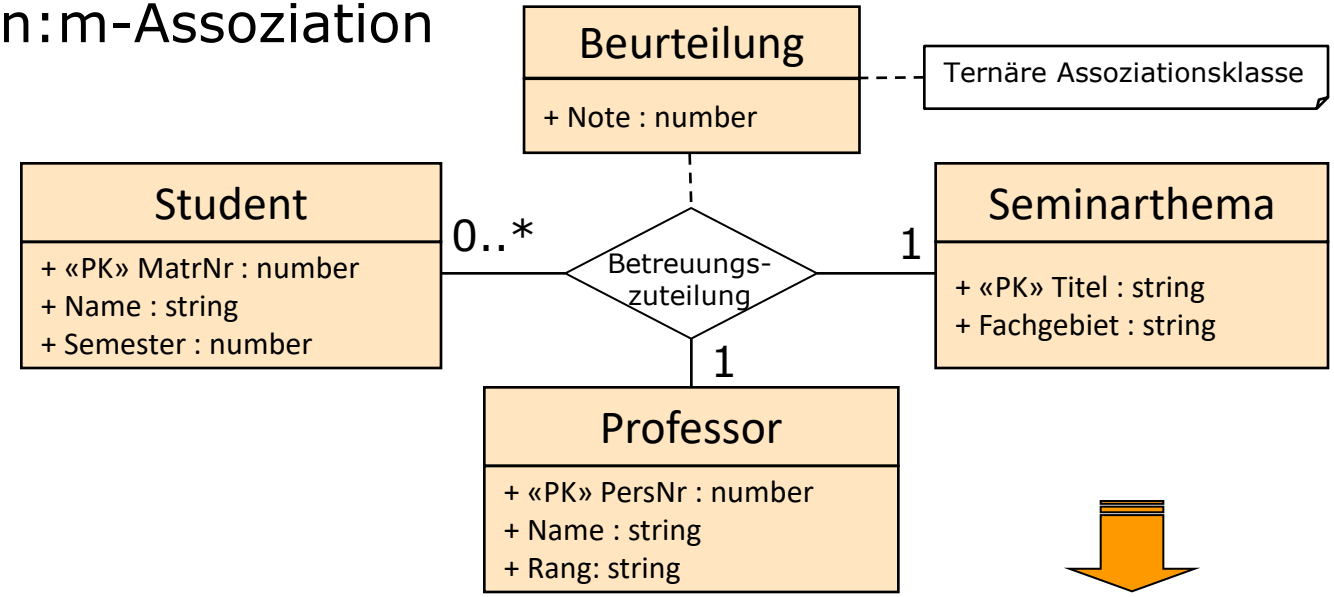
Firma: {[UID:number, Name:string]}

Vertrag: {[SVNR:number, UID:number, Gehalt:number]}

# Abbildung konzeptuelles in logisches Schema

## Abbildung Beziehungstypen – Ternäre Assoziation

Ternäre Beziehung wird zu eigener Relation analog den Regeln einer n:m-Assoziation




**Logischer Entwurf:**

Student: {[MatrNr:number, Name:string, Semester:number]}

Professor: {[PersNr:number, Name:string, Rang:string]}

Seminarthema: {[Titel:string, Fachgebiet:string]}

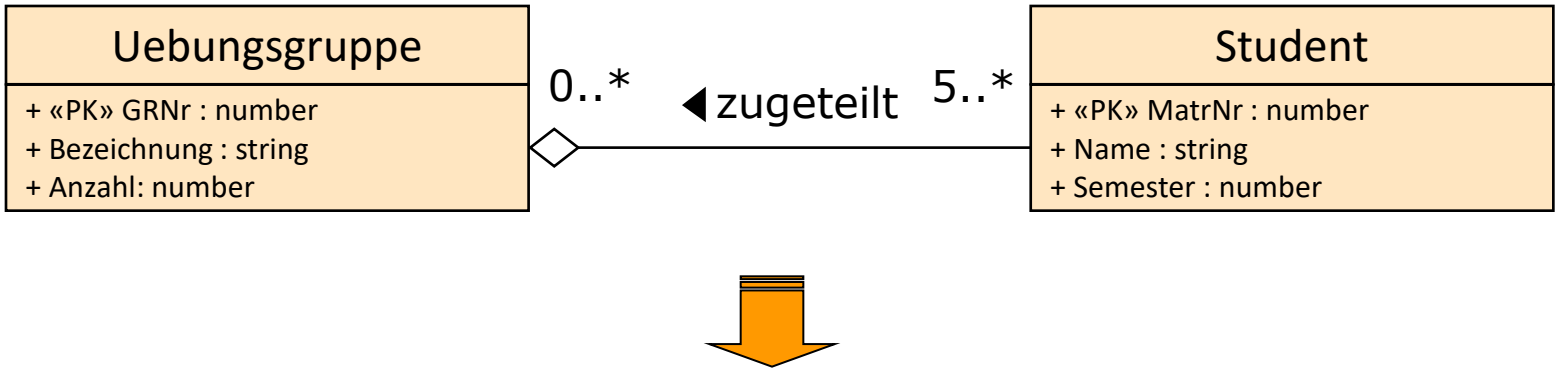
Beurteilung: {[MatrNr:number, PersNr:number, Titel:string, Note:number]}

© 2026

# Abbildung konzeptuelles in logisches Schema

## Abbildung Beziehungstypen – Aggregation

Aggregation wird nach den Beziehungsregeln abgebildet




**Logischer Entwurf:**

Uebungsgruppe: {[GRNr:number, Bezeichnung:string, Anzahl:number]}

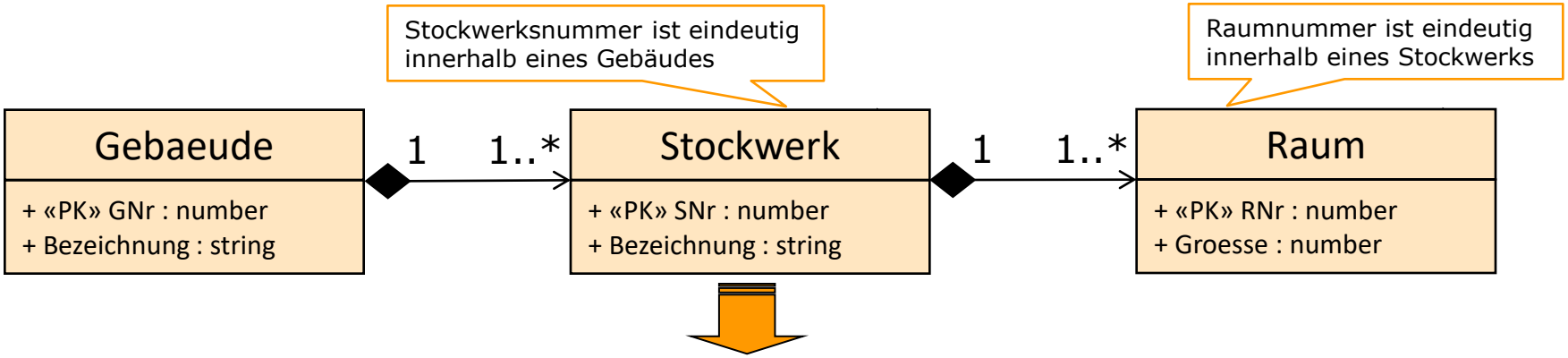
Student: {[MatrNr:number, Name:string, Semester:number]}

zugeteilt: {[MatrNr:number, GRNr:number]}

# Abbildung konzeptuelles in logisches Schema

## Abbildung Beziehungstypen – Komposition

Komposition wird grundsätzlich auch nach den Beziehungssregeln abgebildet, einzig mit dem Unterschied, dass sich der Primärschlüssel der aggregierten Relation aus dem Schlüssel der aggregierenden Relation mit dem Schlüssel der aggregierten Relation zusammensetzt (außer die aggregierte Relation besitzt einen eigenen eindeutigen - z.B. künstlichen - Schlüssel)




**Logischer Entwurf:**

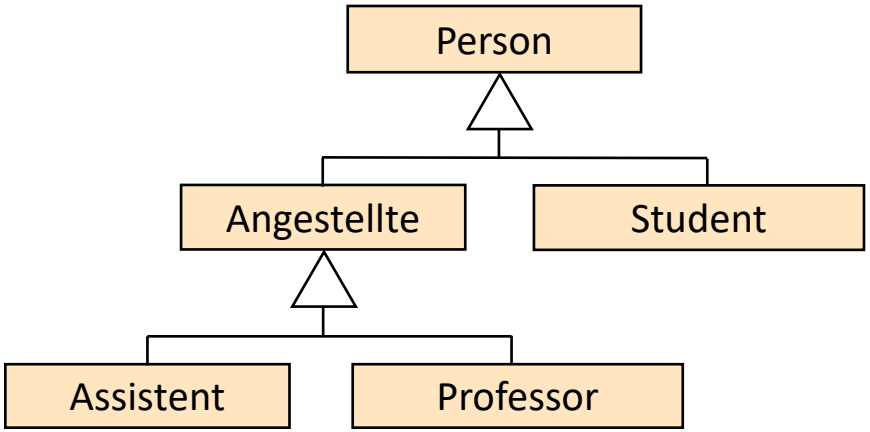
Gebäude: {[GNr:number, Bezeichnung:string]}

Stockwerk: {[GNr:number, SNr:number, Bezeichnung:string]}

Raum: {[GNr:number, SNr:number, RNr:number, Groesse:number]}

© 2026

# Abbildung Generalisierung/Spezialisierung




- Wie können Vererbungshierarchien abgebildet werden?
- Wie werden polymorphe Abfragen durchgeführt (Zugriff auf alle Objekte einer Klasse)?
- Wie kann der konkrete Typ eines Objektes festgestellt werden?
- ...

➢ **Auswahl einer geeigneten Abbildung hängt von konkreter Problemstellung bzw. von den Anforderungen (effiziente Abfragen) ab**

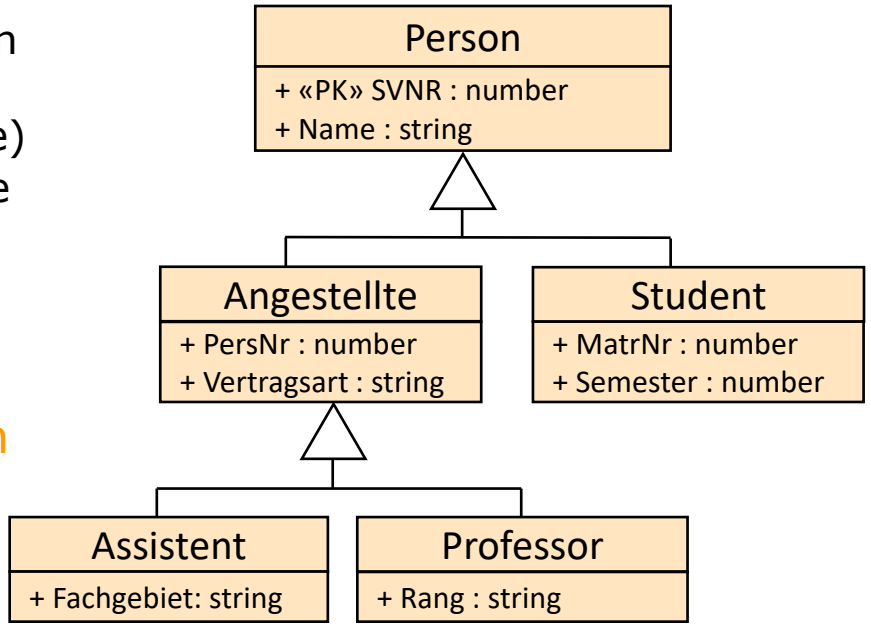
# Überblick

- Datenbankentwurf
  - Abbildung Entitätstypen
  - Abbildung Beziehungstypen
  - Abbildung Generalisierung
- Relationale Modellierung der Generalisierung
  - Abbildungsstrategien/-varianten
    - Basisrelationenmodell
    - Partitionierungsmodell
    - Volle Redundanz
    - Einrelationenmodell
  - Vergleich der Abbildungsstrategien

© 2026

# Abbildung Generalisierung/Spezialisierung

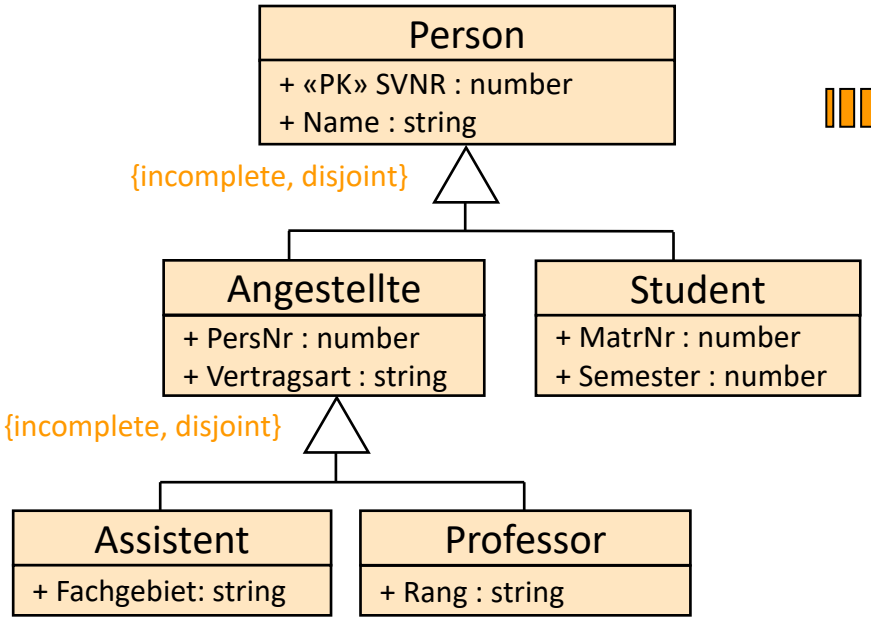
- Relationenmodell bietet keine direkte Unterstützung für das Abstraktionskonzept Generalisierung/Spezialisierung
  - keine Mechanismen für die Vererbung vorhanden (von Struktur, Integritätsbedingungen, Operationen)
- Jedoch **"Simulation" der Generalisierung/Spezialisierung** eingeschränkt möglich (Object-Relational Mapping – ORM)
  - Abhängig von den Anforderungen
    - Speicherverbrauch soll gering sein
    - Einfacher Zugriff auf alle Objekte einer Klasse (polymorphe Abfrage)
    - Einfacher Zugriff auf alle Attribute einer Klasse
    - Konsistenzsicherung
    - Einfache Einfüge- und Aktualisierungsoperationen
  - Abbildungsmöglichkeit auswählen
    - Basisrelationenmodell
    - Partitionierungsmodell
    - Volle Redundanz
    - Einrelationenmodell



# Abbildung Generalisierung/Spezialisierung

## Abbildung von Vererbung: Basisrelationenmodell

- Jede Klasse wird durch eine Relation abgebildet
- Jede Klasse ist genau einmal und vollständig in ihrer Basisrelation gespeichert
- ➢ Es wird eine horizontale Partitionierung der Instanzen erreicht




**Person**

| SVNR | Name |
|---|---|
| 100 | Huber |

**Angestellte**

| SVNR | Name | PersNr | Vertragsart |
|---|---|---|---|
| 200 | Maier | 20200 | A3/2 |

**Student**

| SVNR | Name | MatrNr | Semester |
|---|---|---|---|
| 300 | Jonas | 25403 | 12 |
| 400 | Fichte | 26120 | 10 |

**Assistent**

| SVNR | Name | PersNr | Vertragsart | Fachgebiet |
|---|---|---|---|---|
| 500 | Franz | 20500 | L1/4 | UXD |

**Professor**

| SVNR | Name | PersNr | Vertragsart | Rang |
|---|---|---|---|---|
| 600 | Altmann | 20600 | L2/6 | C3 |
| 700 | Jadin | 20700 | L2/3 | C4 |

© 2026

# Abbildung Generalisierung/Spezialisierung

## Basisrelationenmodell - Bewertung

- minimaler Speicherverbrauch
- keine Änderungsanomalien (setzt voraus, dass Überdeckung disjunkt ist!)
- Zugriff auf alle Attribute einer Klasse erfordert nur eine Abfrage
- Zugriff auf alle Objekte einer Klasse erfordert rekursives Suchen in Unterklassen (Nachfahren)
  - explizite Rekonstruktion durch Relationenoperationen Projektion ($\pi$) und Vereinigung ($\cup$)

**Beispiel**

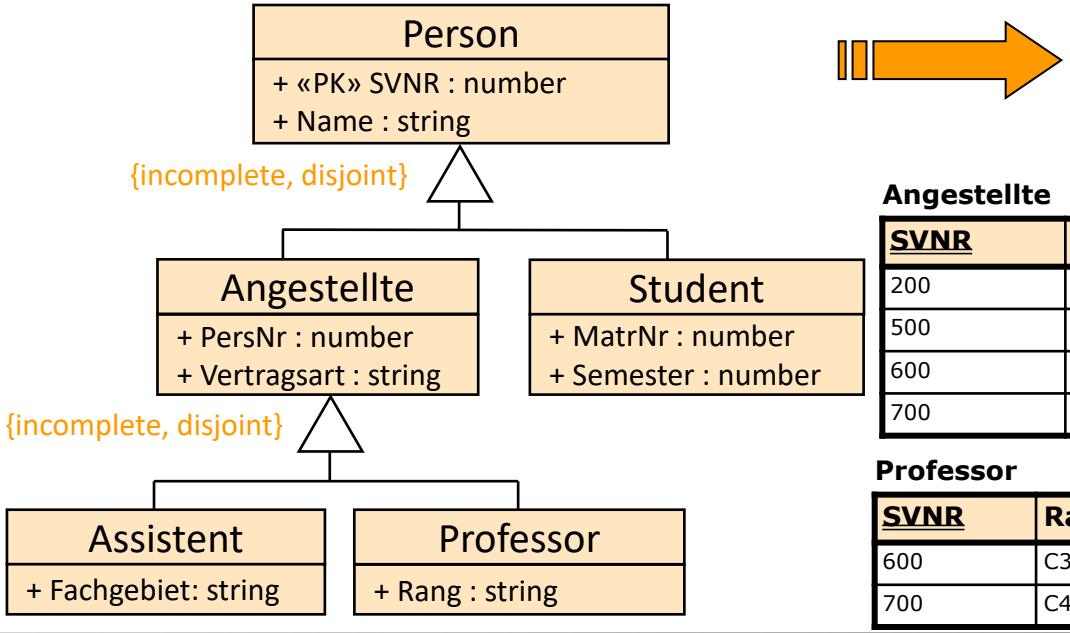
Selektiere alle Angestellten?

$\pi_{SVNR,Name,PersNr}$ (Angestellte) $\cup$ $\pi_{SVNR,Name,PersNr}$ (Assistent) $\cup$ $\pi_{SVNR,Name,PersNr}$ (Professor)

# Abbildung Generalisierung/Spezialisierung

## Abbildung von Vererbung: Partitionierungsmodell

- Jede Klasse wird entsprechend der Attribute in der Vererbungshierarchie zerlegt und in Teilen in den zugehörigen Relationen gespeichert
- Es wird der Primärschlüssel dupliziert
- ➢ Es wird eine vertikale Partitionierung der Instanzen erreicht




**Person**

| SVNR | Name |
|---|---|
| 100 | Huber |
| 200 | Maier |
| 300 | Jonas |
| 400 | Fichte |
| 500 | Franz |
| 600 | Altmann |
| 700 | Jadin |

**Angestellte**

| SVNR | PersNr | Vertragsart |
|---|---|---|
| 200 | 20200 | A3/2 |
| 500 | 20500 | L1/4 |
| 600 | 20600 | L2/6 |
| 700 | 20700 | A3/3 |

**Student**

| SVNR | MatrNr | Semester |
|---|---|---|
| 300 | 25403 | 12 |
| 400 | 26120 | 10 |

**Professor**

| SVNR | Rang |
|---|---|
| 600 | C3 |
| 700 | C4 |

**Assistent**

| SVNR | Fachgebiet |
|---|---|
| 500 | UXD |

# Abbildung Generalisierung/Spezialisierung

## Partitionierungsmodell - Bewertung

- geringer Speicherverbrauch (PK/OID dupliziert)
- Integritätsbedingung:
  - Assistent.SVNR $\subseteq$ Angestellte.SVNR $\subseteq$ Person.SVNR
- Zugriff auf alle Objekte einer Klasse erfordert eine Abfrage
- hohe Aufsuchs- und Aktualisierungskosten
- Zugriff auf alle Attribute einer Klasse erfordert Verbundoperation ($\bowtie$) (alle Vorfahren)

**Beispiel**

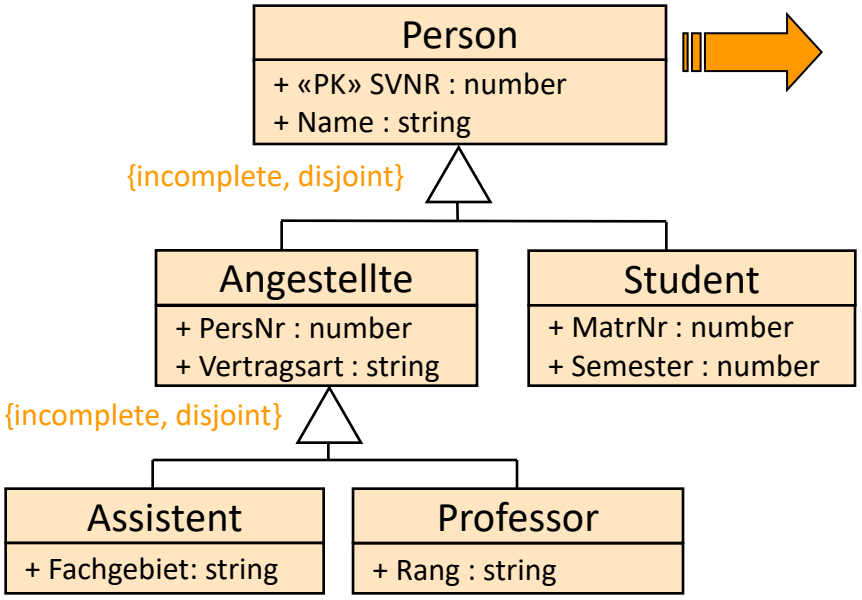
Selektiere alle Attribute zu einem Assistenten?

$$\text{Angestellte} \bowtie \text{Assistent} \bowtie \text{Professor}$$

# Abbildung Generalisierung/Spezialisierung

## Abbildung von Vererbung: Volle Redundanz

- Jede Klasse wird durch eine Relation abgebildet, wobei die geerbten Attribute dupliziert werden (Ausflachung der Vererbungshierarchie) sowie der Primärschlüssel in den Subklassen dupliziert wird
- Dadurch werden die Instanzen dupliziert - sie besitzt dabei die Werte der Attribute, die sie geerbt hat, zusammen mit den Werten der Attribute der konkreten Klasse




**Person**

| SVNR | Name |
|---|---|
| 100 | Huber |
| 200 | Maier |
| 300 | Jonas |
| 400 | Fichte |
| 500 | Franz |
| 600 | Altmann |
| 700 | Jadin |

**Angestellte**

| SVNR | Name | PersNr | Vertragsart |
|---|---|---|---|
| 200 | Maier | 20200 | A3/2 |
| 500 | Franz | 20500 | L1/4 |
| 600 | Altmann | 20600 | L2/6 |
| 700 | Jadin | 20700 | A3/3 |

**Student**

| SVNR | Name | MatrNr | Semester |
|---|---|---|---|
| 300 | Jonas | 25403 | 12 |
| 400 | Fichte | 26120 | 10 |

**Assistent**

| SVNR | Name | PersNr | Vertragsart | Fachgebiet |
|---|---|---|---|---|
| 500 | Franz | 20500 | L1/4 | UXD |

**Professor**

| SVNR | Name | PersNr | Vertragsart | Rang |
|---|---|---|---|---|
| 600 | Altmann | 20600 | L2/6 | C3 |
| 700 | Jadin | 20700 | L2/3 | C4 |

# Abbildung Generalisierung/Spezialisierung

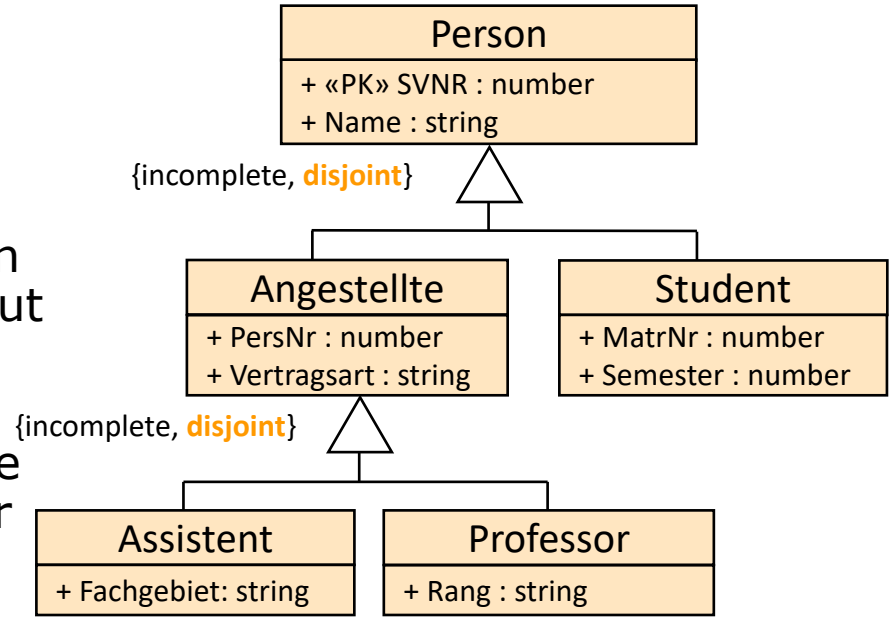
## Volle Redundanz - Bewertung

- sehr hoher Speicherverbrauch
- Auftreten von Änderungsanomalien
- Zugriff auf alle Objekte einer Klasse erfordert eine Abfrage
- Zugriff auf alle Attribute einer Klasse erfordert eine Abfrage

# Abbildung Generalisierung/Spezialisierung

## Abbildung von Vererbung: Einrelationenmodell (disjunkt)

- Eine gesamte Vererbungshierarchie wird durch eine einzige Relation abgebildet
- Um die Typinformation der Instanzen nicht zu verlieren, wird ein Typattribut (Diskriminator) eingeführt
- Bei dieser Abbildungsvariante resultieren viele NULL-Werte, da eine Instanz keine Werte für Attribute der Subklassen aufweisen kann




**Person**

| SVNR | Name | PersNr | Vertragsart | MatrNr | Semester | Fachgebiet | Rang | PersonTyp |
|---|---|---|---|---|---|---|---|---|
| 100 | Huber | NULL | NULL | NULL | NULL | NULL | NULL | Person |
| 200 | Maier | 20200 | A3/2 | NULL | NULL | NULL | NULL | Angestellte |
| 300 | Jonas | NULL | NULL | 25403 | 12 | NULL | NULL | Student |
| 400 | Fichte | NULL | NULL | 26120 | 10 | NULL | NULL | Student |
| 500 | Franz | 20500 | L1/4 | NULL | NULL | UXD | NULL | Assistent |
| 600 | Altmann | 20600 | L2/6 | NULL | NULL | NULL | C3 | Professor |
| 700 | Jadin | 20700 | L2/3 | NULL | NULL | NULL | C4 | Professor |

Ungenutzte Attribute

Klassenkennzeichen

© 2026

# Abbildung Generalisierung/Spezialisierung

## Volle Redundanz – Einrelationenmodell (disjunkt)

- Einführen eines Typ-Attributes (Klassenkennzeichen)
- erzeugt viele Nullwerte, wenn Untertypen viele spezifische Attribute besitzen
- Zugriff auf alle Objekte einer Klasse erfordert eine Abfrage (Bedingung auf Typ-Attribut)
- Zugriff auf alle Attribute einer Klasse erfordert eine Abfrage (Bedingung auf Typ-Attribut)

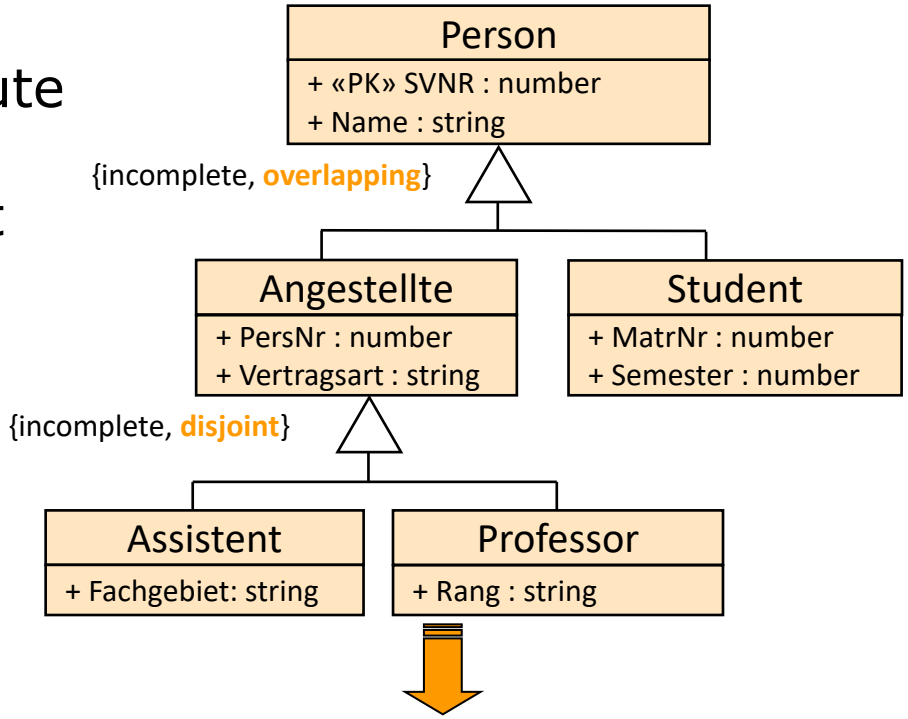
- ist anderen Abbildungen vorzuziehen, wenn
  - Untertypen wenige spezifische Attribute besitzen
    ⇨ wenige Nullwerte
  - Vererbungshierarchie flach ist
    ⇨ kleine Tabellen
  - polymorphe Abfragen benötigt werden

© 2026

# Abbildung Generalisierung/Spezialisierung

## Abbildung von Vererbung: Einrelationenmodell (überlappend)

- Mehrere boolesche Typ-Attribute bestimmen, zu welchen Untertypen eine Entität gehört




**Person**

| SVNR | Name | AngestellteTyp | PersNr | Vertragsart | Fachgebiet | Rang | StudentTyp | MatrNr | Semester |
|---|---|---|---|---|---|---|---|---|---|
| 100 | Huber | false | NULL | NULL | NULL | NULL | false | NULL | NULL |
| 200 | Maier | true | 20200 | A3/2 | NULL | NULL | false | NULL | NULL |
| 300 | Jonas | false | NULL | NULL | NULL | NULL | true | 25403 | 12 |
| 400 | Fichte | true | 20400 | Tutor/2 | NULL | NULL | true | 26120 | 10 |
| 500 | Franz | true | 20500 | L1/4 | UXD | NULL | false | NULL | NULL |
| 600 | Altmann | true | 20600 | L2/6 | NULL | C3 | false | NULL | NULL |
| 700 | Jadin | true | 20700 | L2/3 | NULL | C4 | false | NULL | NULL |

# Zusammenfassung

## Vergleich der Abbildungsarten/-varianten



| | Speicher-verbrauch | Zugriff auf alle Objekte einer Klasse | Zugriff auf alle Attribute einer Klasse | Inserts/Updates |
|---|---|---|---|---|
| **Basisrelationenmodell (horizontale Partitionierung)** | + Minimal | - Union mit Projektion über alle Nachfahren | + 1 Abfrage | + 1 Update |
| **Partitionierungsmodell (vertikale Partitionierung)** | + Nur Duplizierung des PKs | + 1 Abfrage: OID + wenige Attribute | - Verbund über alle Vorfahren | - Im Worst Case N Updates über alle Vorfahren nötig |
| **Volle Redundanz** | - Duplizierung der Instanzen | + 1 Abfrage | + 1 Abfrage | - N Updates über alle Vorfahren |
| **Einrelationenmodell** | ~ NULL-Werte | + 1 Abfrage mit Bedingung auf Typ-Attribut | + 1 Abfrage mit Bedingung auf Typ-Attribut | + 1 Update |

**Optimale Abbildung ist anwendungsabhängig!**

© 2026